CHAPTER 3 · TRANSPORT LAYER

3.5

Connection-Oriented Transport: TCP

Kurose & Ross, 8th Ed.

INTRO

Chapter 3.5 — Roadmap

Six subsections, one complete picture of TCP

1. **3.5.1 The TCP Connection**
   Connection-oriented, point-to-point, full-duplex, MSS, buffers.

2. **3.5.2 TCP Segment Structure**
   Header fields: SeqNum, AckNum, flags, receive window — every bit explained.

3. **3.5.3 RTT Estimation & Timeout**
   EWMA, EstimatedRTT, DevRTT, TimeoutInterval formula, exponential backoff.

4. **3.5.4 Reliable Data Transfer**
   Pipelining, single timer, three sender events, fast retransmit.

5. **3.5.5 Flow Control**
   Receive window (rwnd), sender constraint, zero-window deadlock & fix.

6. **3.5.6 Connection Management**
   Three-way handshake, four-way teardown, SYN flood, SYN cookies.

SECTION 3.5.1

The TCP Connection

Point-to-point · Full-duplex · MSS · Buffers at endpoints only

3.5.1

Fundamental Properties of a TCP Connection

Section 3.5.1 — The TCP Connection

Point-to-Point

Always exactly ONE sender and ONE receiver.

No multicasting possible in TCP.

"Two hosts are company — three are a crowd."

Full-Duplex

Data flows A→B and B→A simultaneously over the same connection.

Each side is simultaneously sender AND receiver.

This is why the header has both SeqNum AND AckNum.

Connection-Oriented

Before data flows, both sides must handshake and initialize state variables.

Only after this setup can application data be exchanged.

TCP connection state = buffers + variables at both endpoints ONLY.
No state in routers, switches, or any network element.

End-to-end principle: complexity lives at the edges, not in the network.

3.5.1

MSS, Buffers, and the Data Flow Model

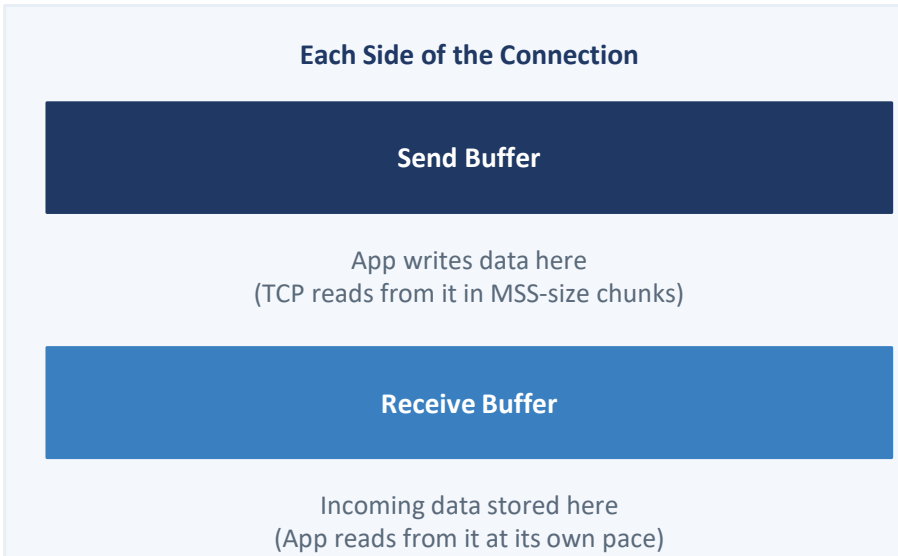
Section 3.5.1 — The TCP Connection

Maximum Segment Size (MSS)

MSS = maximum APPLICATION DATA per segment.
(NOT total segment size including headers.)

Typical value: 1,460 bytes (Ethernet)
Set by the local link-layer MTU.

Large file: broken into chunks of MSS bytes.
Each chunk + TCP header = one TCP segment.



No buffers are allocated in any router or switch — the connection state lives only at the two endpoints.

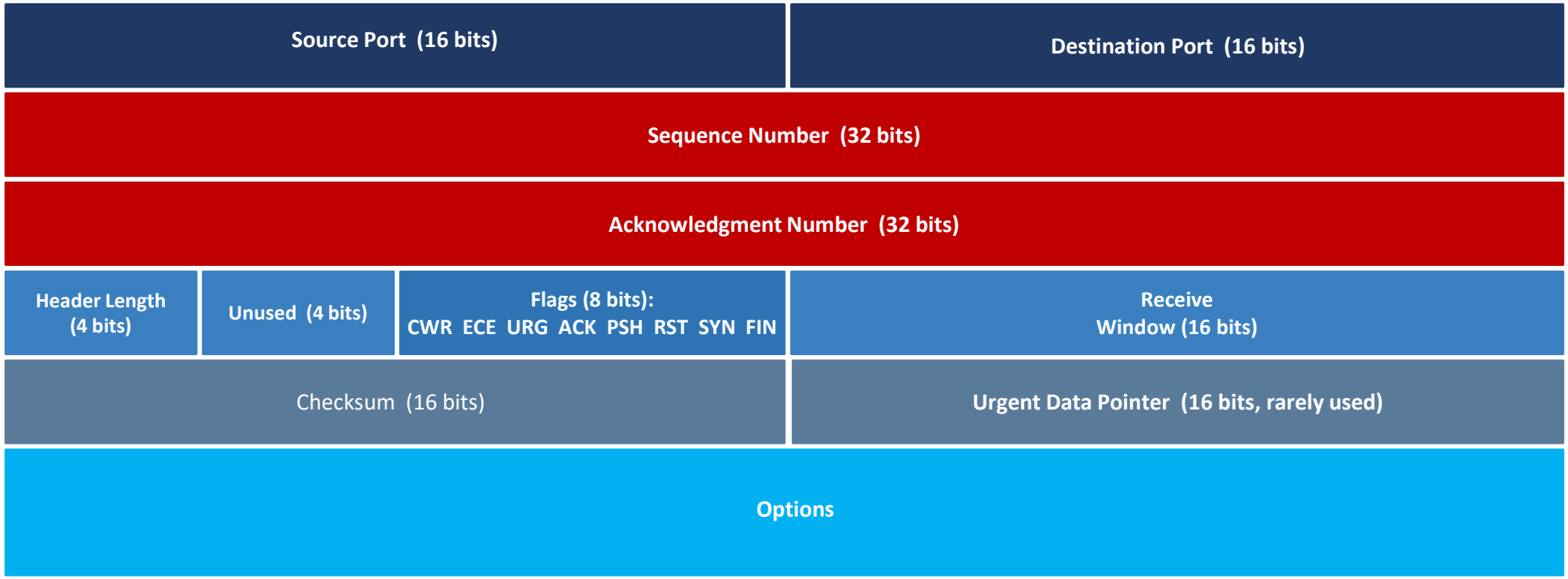
SECTION 3.5.2

TCP Segment Structure

20-byte header · SeqNum · AckNum · Flags · Receive Window

TCP Segment Header — All Fields

Section 3.5.2 — Segment Structure



Standard TCP header = 20 bytes (no options)

3.5.2

TCP Header Fields — Role of Each Field

Section 3.5.2 — Segment Structure

Source Port & Destination Port (16 + 16 bits)

Identify the sending and receiving APPLICATION processes. Together with IP addresses they form the 4-tuple that uniquely identifies a TCP connection. Used by the OS to demultiplex incoming segments to the correct socket.

Sequence Number (32 bits)

Byte-stream position of the FIRST byte of data in this segment. Enables the receiver to reassemble segments in order and detect gaps. Supports cumulative acknowledgment.

Acknowledgment Number (32 bits)

The sequence number of the NEXT byte the sender of this segment expects to receive. Cumulative: acknowledges all bytes up to (but not including) this number. Valid only when ACK flag = 1.

Checksum (16 bits)

Error detection over the entire segment (header + data). Corrupted segments are silently discarded. Same mechanism as UDP checksum. Computed over a pseudo-header including IP addresses.

Receive Window (16 bits)

Used for FLOW CONTROL. Tells the remote sender how many bytes of buffer space are currently available at this receiver. The sender must not have more unacknowledged bytes in flight than this value.

Header Length (4 bits)

Length of the TCP header in 32-bit words. Needed because the Options field makes the header variable-length. Minimum value = 5 (= 20 bytes, no options). Maximum = 15 (= 60 bytes).

Options (variable, 0 – 40 bytes)

TLV-encoded extensions. Common options:
- MSS — max segment size negotiation
- Window Scale — extend rwnd >65535
- Timestamps — precise RTT (RFC 7323)

Urgent Data Pointer (16 bits)

Valid only when URG=1. Points to the last byte of urgent data. Rarely used in modern applications — originally for Telnet Ctrl+C out-of-band signaling.

TCP Flag Bits — Role of Each Flag

Section 3.5.2 — Segment Structure

The 8-bit Flags field controls TCP connection state.
Each bit has a distinct role — only ACK is set in nearly every segment.

FIN (bit 0)

Final — No More Data

Sent when a side has finished transmitting data.

Initiates half-close: sender can no longer send, but can still receive. Four-way teardown requires one FIN from each side.

SYN (bit 1)

Synchronize Sequence Numbers

Set only during connection setup (SYN and SYNACK segments).

Carries the sender's Initial Sequence Number (ISN). Randomly chosen to prevent confusion with old connections and ISN-prediction attacks.

RST (bit 2)

Reset — Abort Connection

Sent on a segment to a closed port, or to forcefully terminate.

No graceful close — all buffered data is discarded immediately. The receiver releases connection resources at once.

PSH (bit 3)

Push — Deliver Immediately

Hints to the receiver to flush buffered data to the application.

Most implementations ignore PSH and deliver data at their own pace. Rarely needs to be set explicitly by application code.

ACK (bit 4)

Acknowledgment Field Valid

Set in every segment AFTER the initial SYN.

When ACK=1, the AckNum field is meaningful. Without it, the receiver ignores the acknowledgment number entirely.

URG (bit 5)

Urgent Data Present

Indicates the Urgent Pointer field is valid.

Rarely used in modern protocols. Originally designed for Telnet Ctrl+C. Applications now use application-level out-of-band signaling.

ECE (bit 6)

ECN-Echo (Congestion Notification)

Used in Explicit Congestion Notification (RFC 3168).

Signals to the TCP sender that a CE-marked packet was received. Allows routers to signal congestion without causing packet loss.

CWR (bit 7)

Congestion Window Reduced

Sent by the TCP sender in response to an ECE signal.

Acknowledges receipt of the ECN-Echo: sender has already reduced its congestion window. Prevents duplicate congestion responses.

Sequence Numbers — Byte-Stream Numbering

3.5.2

Section 3.5.2 — Segment Structure

TCP numbers BYTES, not segments.
SeqNum = byte-stream position of the FIRST BYTE in the segment.

Segment 1	Segment 2	Segment 3
bytes 0 – 999	bytes 1000 – 1999	bytes 2000 – 2999
`SeqNum = 0`	`SeqNum = 1,000`	`SeqNum = 2,000`

File: 500,000 bytes | MSS = 1,000 bytes → 500 segments

Initial Sequence Number (ISN)

Chosen RANDOMLY at connection setup.

Reason 1: Security — predictable ISN enables forged segments.
Reason 2: Prevents old segments from stale connections being accepted as valid.

Cumulative Acknowledgments

AckNum = next byte expected.
If bytes 0–535 received: AckNum = 536.
Out-of-order bytes are buffered;
ACK stays at the first gap until it is filled.

3.5.2

Seq & Ack Numbers — Telnet Case Study

Section 3.5.2 — Piggybacking

User types 'C'. Client ISN = 42. Server ISN = 79. Watch how SeqNum and AckNum evolve.

Direction	SeqNum	AckNum	Data	Meaning
A → B	42	79	'C' (1 byte)	A sends 'C'; expects byte 79 from B
B → A	79	43	'C' (echo back)	B ACKs byte 42; echoes 'C' back (PIGGYBACKING)
A → B	43	80	(empty)	A ACKs the echo; no new data

Segment 2 carries both data ('C' echo) AND an acknowledgment — this is PIGGYBACKING.

SECTION 3.5.3

RTT Estimation & Timeout

EWMA · EstimatedRTT · DevRTT · TimeoutInterval · Exponential backoff

Why RTT Estimation Is Hard

Section 3.5.3 — Round-Trip Time Estimation

Timeout must be longer than RTT — but RTT varies.
Too short → unnecessary retransmits. Too long → slow recovery.

SampleRTT

Time from segment sent to ACK received.

Measured for only ONE segment at a time (not for retransmitted segments — Karn's algorithm).

Fluctuates with congestion and load.

EstimatedRTT (EWMA)

Exponential Weighted Moving Average:

$EstRTT = (1-\alpha) \times EstRTT + \alpha \times SampleRTT$

$\alpha = 0.125$ (recommended, RFC 6298)

Recent samples weighted MORE.
Smooths out short-term fluctuations.

DevRTT (Variability)

Estimates how much SampleRTT varies:

$DevRTT = (1-\beta) \times DevRTT + \beta \times |SampleRTT - EstRTT|$

$\beta = 0.25$ (recommended)

Large when RTT is volatile.
Small when RTT is stable.

3.5.3

Setting the Timeout Interval

Section 3.5.3 — Round-Trip Time Estimation

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$$

Interpreting the Formula

EstimatedRTT is the baseline.
4×DevRTT is the safety margin.

When RTT is stable (DevRTT small):
timeout tracks RTT closely.

When RTT fluctuates (DevRTT large):
larger margin prevents spurious timeouts.

Initial value: 1 second (RFC 6298).

Exponential Backoff on Timeout

When a timeout fires:
TimeoutInterval is DOUBLED
before retransmitting.

This is the beginning of TCP's congestion response — back off when the network seems troubled.

As soon as a new ACK arrives, TimeoutInterval is recomputed using the formula.

```
Example: EstRTT=100ms, DevRTT=10ms  →  TimeoutInterval = 100+4×10 = 140ms
After spike (SampleRTT=200ms): EstRTT = 0.875×100 + 0.125×200 = 112.5ms
```

SECTION 3.5.4

Reliable Data Transfer

Pipelining · Single timer · Three sender events · Fast retransmit

3.5.4

TCP Sender — Three Key Events

Section 3.5.4 — Reliable Data Transfer

TCP uses pipelining with a SINGLE retransmission timer covering all outstanding unacknowledged segments.

1

Data from Application

Create a segment with the next sequence number.
Pass to IP. If timer not running, start it.
(Timer ticks for the OLDEST unacknowledged segment.)

2

Timer Timeout

Retransmit the segment that caused the timeout (oldest unACKed).
Restart the timer.
DOUBLE the TimeoutInterval (exponential backoff).

3

ACK Received

If ACK acknowledges new data: update SendBase.
If outstanding unACKed segments remain: restart timer.
If all data acknowledged: stop the timer.

3.5.4

Fast Retransmit — Don't Wait for the Timer

Section 3.5.4 — Reliable Data Transfer

If 3 duplicate ACKs arrive for the same data, retransmit the missing segment IMMEDIATELY — before the timer expires.

Why 3 duplicates? Fewer could be normal reordering. Three is a strong signal of loss — not a false positive.

TCP Reliability — Five Mechanisms Working Together

Section 3.5.4 — Reliable Data Transfer

Mechanism 1 — Checksum

Detects corrupted segments.
Corrupted segments are silently discarded.
TCP then retransmits them via timeout.

Mechanism 2 — Sequence Numbers

Identify byte positions in the stream.
Detect gaps (lost segments) and reordering.
Enable the receiver to reassemble correctly.

Mechanism 3 — Cumulative ACKs

Acknowledge all bytes up to the first gap.
Out-of-order bytes are buffered, not discarded.
ACK stays at the gap until it is filled.

Mechanism 4 — Timer + Backoff

Single timer covers all outstanding segments.
Timeout → retransmit oldest unACKed segment.
Timeout interval doubles on each timeout.

Mechanism 5 — Fast Retransmit

3 duplicate ACKs → retransmit immediately.
Faster recovery than waiting for the timer.
Pipelining ensures the pipe stays full.

The TCP Guarantee

Byte stream received by application:
✓ No corruption ✓ No gaps ✓ No duplicates ✓ In order

SECTION 3.5.5

Flow Control

rwnd · Receiver buffer · Sender constraint · Zero-window deadlock

3.5.5

Flow Control — Speed Matching

Section 3.5.5 — Flow Control

Flow Control: prevent the SENDER from overflowing the RECEIVER'S buffer.

The Problem

Fast sender + slow application reader = buffer overflow.

Example:
- Network: 1 Gbps
- App reads at: 10 Mbps

Without control: buffer fills in milliseconds.
Data dropped. Retransmissions. Waste.

Flow Control vs. Congestion Control

Flow control:
- Is the RECEIVER'S BUFFER overwhelmed?
- Controlled by: rwnd (receive window)

Congestion control (Section 3.7):
- Is the NETWORK overwhelmed?
- Controlled by: cwnd (congestion window)

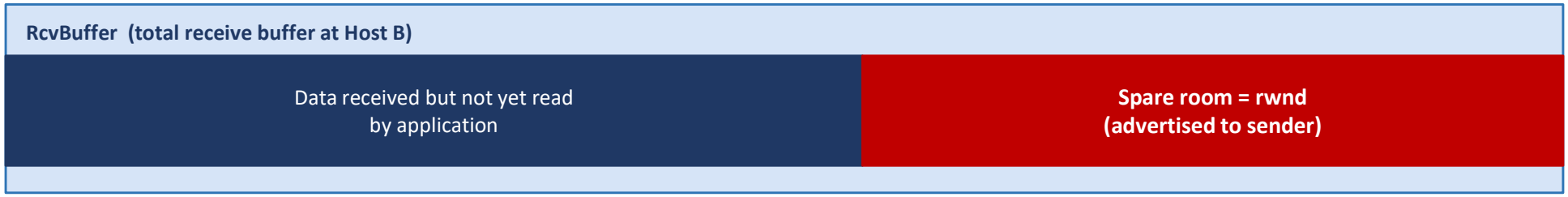
Same throttling mechanism, different problem.

```
Sender constraint:  LastByteSent – LastByteAcked  ≤  rwnd

rwnd = RcvBuffer – (LastByteRcvd – LastByteRead)

(rwnd is dynamic: grows as app reads, shrinks as data arrives)
```

3.5.5

The Receive Window (rwnd) Mechanism

Section 3.5.5 — Flow Control



How rwnd Is Communicated

Host B places its current rwnd value in the Receive Window field of EVERY TCP segment it sends to Host A.

Host A reads this value and adjusts: keeps unACKed bytes ≤ rwnd.

rwnd is updated continuously throughout the connection.

The Zero-Window Deadlock

Buffer full → B advertises rwnd=0.
B has nothing to send → never sends update.
A waits forever.

FIX: A sends 1-byte probe segments periodically.
B ACKs them, including current rwnd.
When buffer drains: non-zero rwnd in ACK.
A resumes.

1-byte probe segments prevent the zero-window deadlock — a simple fix for what would otherwise be a fatal hang.

SECTION 3.5.6

Connection Management

Three-way handshake · Four-way teardown · SYN flood · SYN cookies

Establishing a Connection: Three-Way Handshake

Section 3.5.6 — Connection Management

**Three segments are exchanged before any application data can flow.
Both sides initialize sequence numbers and allocate buffers.**

1 **Client sends SYN**

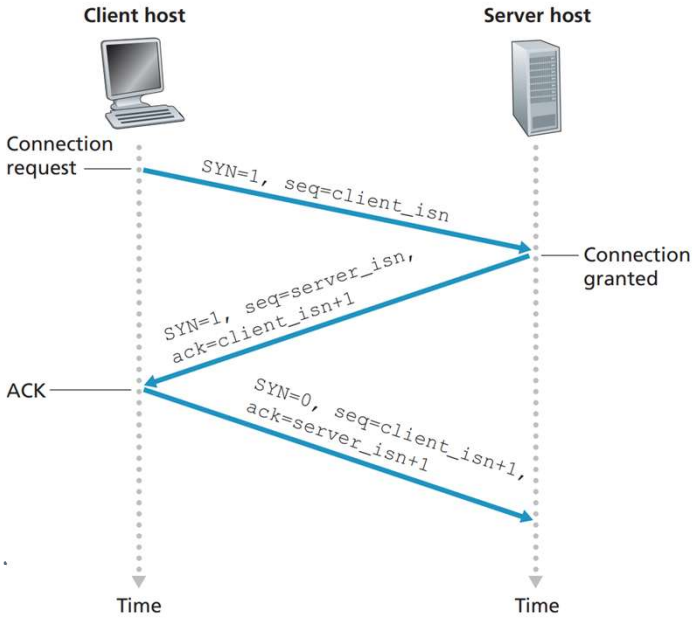
Client TCP sends a special segment: SYN=1, no application data.
Client randomly chooses an initial sequence number: seq = client_isn
Client enters SYN_SENT state.

2 **Server responds with SYNACK**

Server extracts SYN, allocates TCP buffers and variables.
Sends SYNACK segment: SYN=1, ACK=1, ack = client_isn+1, seq = server_isn.
Server enters SYN_RCVD state. Message: "I agree — here is my ISN."

3 **Client sends ACK + data**

Client allocates its own buffers and variables.
Sends ACK segment: SYN=0, ACK=1, ack = server_isn+1. May carry first application data.
Connection is now ESTABLISHED on both sides — data can flow.



Three segments total. Random ISNs: (1) prevent forged segments; (2) prevent stale segments from old connections being accepted.

Closing a Connection: Four-Way Teardown

Section 3.5.6 — Connection Management

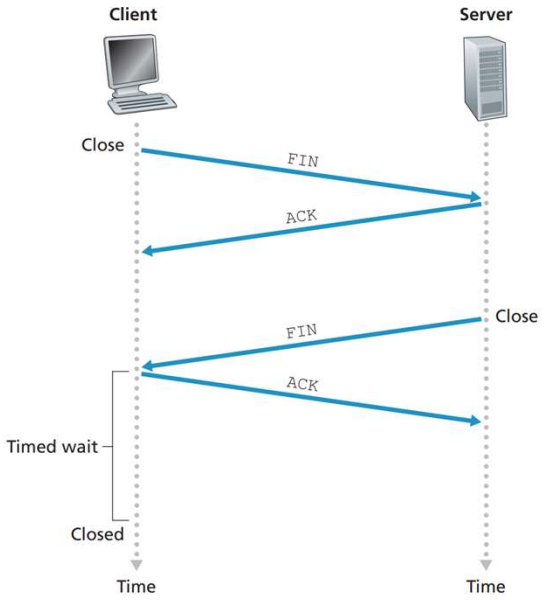
Either side can initiate teardown. Each side closes its half of the connection independently.

1 **Client sends FIN**
FIN=1. Client enters FIN_WAIT_1 state. Client has no more data to send.

2 **Server ACKs**
Server acknowledges. Client enters FIN_WAIT_2. Server can still send data (half-closed).

3 **Server sends FIN**
Server is also done. Sends its own FIN=1. Enters CLOSE_WAIT, then LAST_ACK.

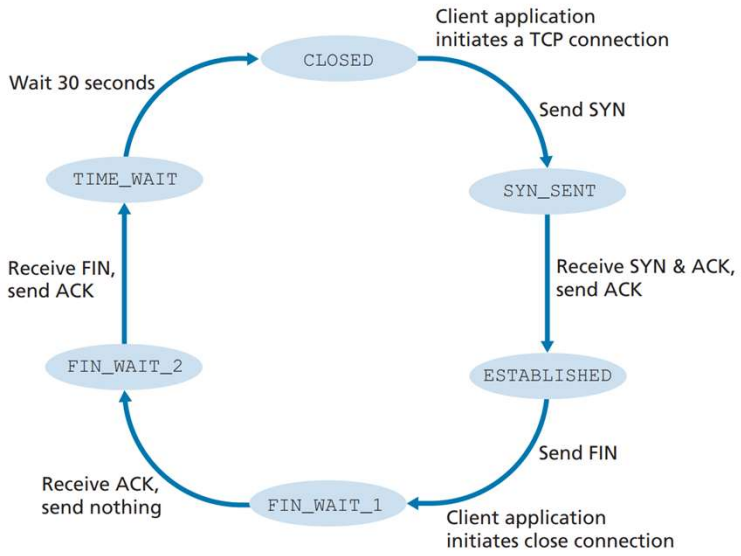
4 **Client ACKs + TIME_WAIT**
Client ACKs server's FIN. Enters TIME_WAIT (30s – 2 min) before full close.
TIME_WAIT protects against lost final ACK — allows retransmission if needed.



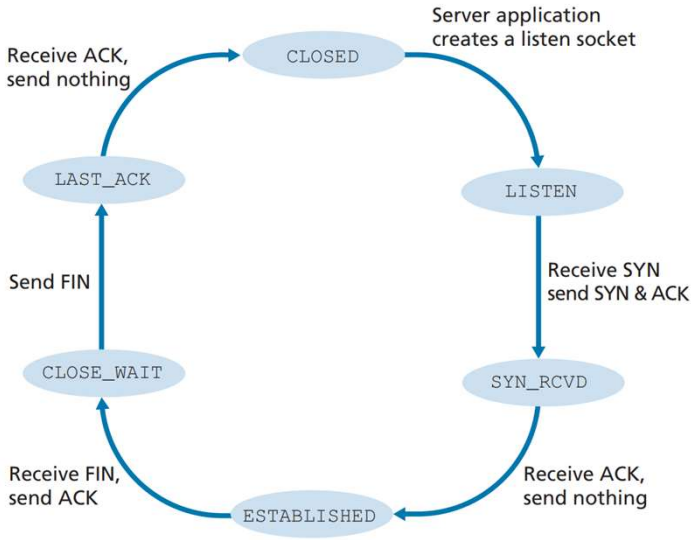
After TIME_WAIT: all resources (buffers, port numbers) are released. Four segments total for a clean close.

TCP State Machine

Section 3.5.6 — Connection Management



TCP State machine - Client Side



TCP State machine - Server Side

These two diagrams assume the client begins connection teardown.

SECURITY

SYN Flood Attack — DoS via Half-Open Connections

Section 3.5.6 — Connection Management Security

The server allocates buffers & variables on receipt of each SYN — BEFORE the handshake completes. This is the vulnerability.

The Attack

Attacker sends thousands of SYN segments with spoofed source IPs.

For each SYN: server allocates resources + sends SYNACK.

Third ACK never arrives (attacker ignores SYNACK).

Server accumulates thousands of HALF-OPEN connections.

Server runs out of memory.
Legitimate connections are REFUSED.

SYN Cookie Defense

On receiving SYN — allocate NOTHING.

Instead, compute a COOKIE:
cookie = hash(src_IP, dst_IP, src_port, dst_port, secret)

Send SYNACK with cookie as server ISN.

Legitimate client sends ACK:
ack = cookie + 1
Server recomputes hash, verifies.
ONLY NOW allocate resources.

Bogus SYN: zero resources used.

**SYN cookies are now standard in all major OS implementations.
First documented DoS attack (1996) — now fully mitigated.**

SYNTHESIS

Everything Together: One HTTP Request over TCP

Chapter 3.5 — End-to-End Synthesis

Every time you click a link, this entire choreography happens — often in under 100 milliseconds.

1. **3-Way Handshake**
   SYN → SYNACK → ACK. Both sides init buffers, ISNs, and state variables.
2. **Segmentation**
   Browser sends HTTP GET. TCP breaks it into MSS-size chunks with sequence numbers.
3. **Piggybacking**
   Each segment carries ACKs for data received from the other side.
4. **Adaptive Timeout**
   TCP tracks EstimatedRTT and DevRTT. Timeout = EstRTT + 4×DevRTT. Doubles on timeout.
5. **Fast Retransmit**
   3 duplicate ACKs → immediate retransmit without waiting for timer to expire.
6. **Flow Control**
   Server advertises rwnd every segment. Browser keeps unACKed bytes ≤ rwnd.
7. **4-Way Teardown**
   FIN → ACK → FIN → ACK + TIME_WAIT. All resources released.

Summary

3.5 — Connection-Oriented Transport: TCP

- TCP is point-to-point, full-duplex, connection-oriented. State (buffers + variables) lives ONLY at the two endpoints.
- MSS = max application data per segment (~1460 bytes). Each side has independent send and receive buffers.
- Sequence numbers label BYTES not segments. AckNum = next expected byte. Cumulative ACKs.
- Standard header: 20 bytes. Key flags: SYN (setup), FIN (teardown), ACK (always after SYN), RST (abort).
- EstimatedRTT = (1-α)×EstRTT + α×SampleRTT (α=1/8). DevRTT tracks variability (β=1/4).
- TimeoutInterval = EstimatedRTT + 4×DevRTT. Doubles on every timeout (exponential backoff).
- Single retransmission timer. Three events: data received, timeout, ACK. Fast retransmit on 3 dupACKs.
- Flow control: sender keeps LastByteSent – LastByteAcked ≤ rwnd (advertised by receiver in every segment).
- Zero-window deadlock fixed by 1-byte probe segments sent by the sender when rwnd=0.
- Three-way handshake: SYN → SYNACK → ACK. Random ISNs prevent forgery and stale segment confusion.
- SYN flood: fill server with half-open connections. SYN cookies: hash-based ISN, no state until ACK verified.